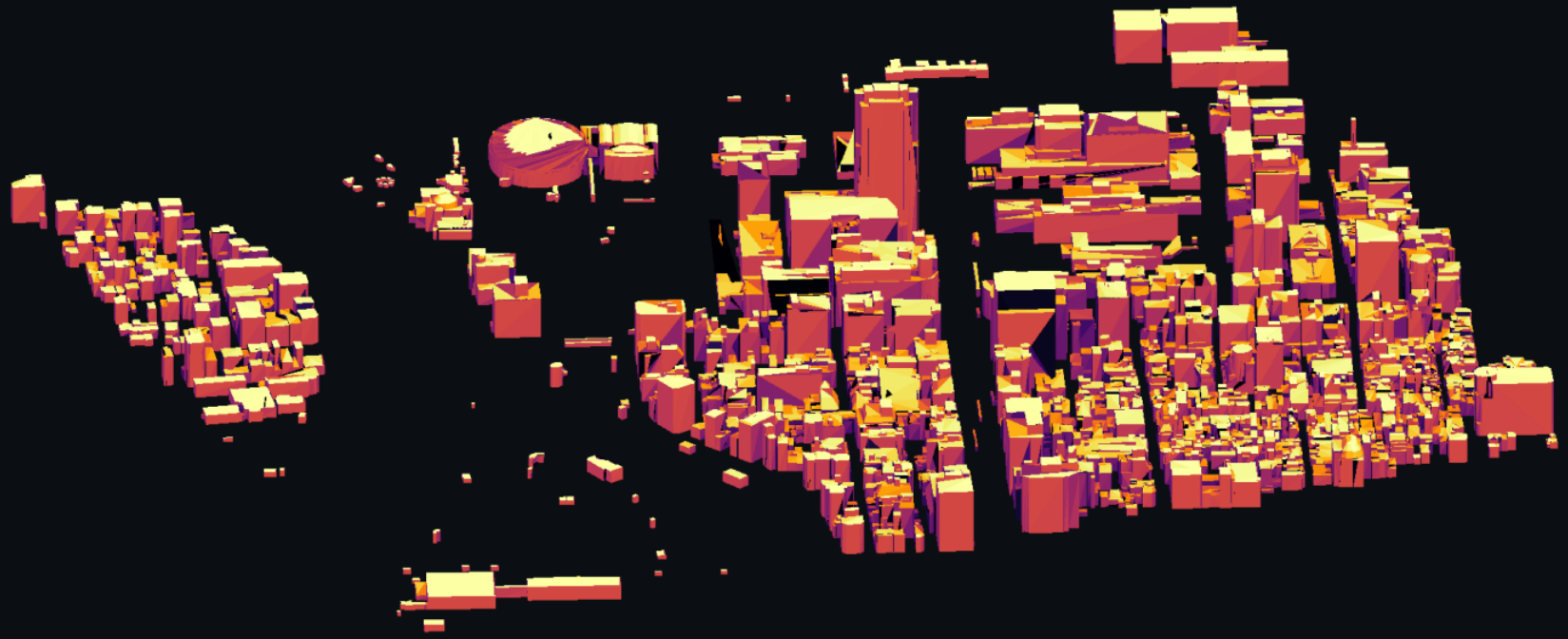
都市のなかで光を計算する

Neuro3D 都市日射エンジン ／ PLATEAU CityGML LOD2

ビューア



街区まるごと 44,902 面の年間日射量を 40 秒。パストレーサとの一致 R^2 0.985（開けた面）。

要旨

PLATEAU の街区をそのまま入力に、面ごとの数値を返します

44,902	40 秒	0.985	26 倍
計算した面の数	光輸送の時間	対照との一致 R^2	同じ負荷での速さ

- 屋上太陽光の適地抽出：屋根面ごとの年間日射量をそのまま並べ替えるだけ。
- 日影規制・再開発の事前検討：街区を丸ごと、設計案ごとに数十秒で解き直せます。
- 緑化・広場・ベンチの配置：天空率と受熱の分布から、夏に暑くなる面がわかります。
- 研究と教育：コードは Apache-2.0、結果はブラウザでそのまま触れます。

入力は PLATEAU の CityGML だけ。追加の測量も、商用ソフトも、GPU も要りません。

リポジトリ

解決する課題

全国 300 都市以上の LOD2 が開いているのに、街区をまるごと解く計算のしかたが足りていません

- 日射と日影は都市計画の基礎データですが、専門ツールは面ごとに光線を追うため、
  街区を超えると計算時間が急に伸びます。
- 自治体が屋上太陽光の適地を探すにも、日影規制を事前に検討するにも、設計案ごとに
  街区を解き直せる速さが要ります。
- 既存の手順は商用ソフトや GPU を前提にしがちで、再現も検証も外から行えません。

本作は、この計算を光線追跡ではなく描画に置き換えます。入力は PLATEAU
の CityGML だけ。出力は面ごとの年間日射量と天空率、そして屋上太陽光の候補一覧です。
コードも対照も失敗した反復も公開し、だれでも同じ数字を出し直せます。

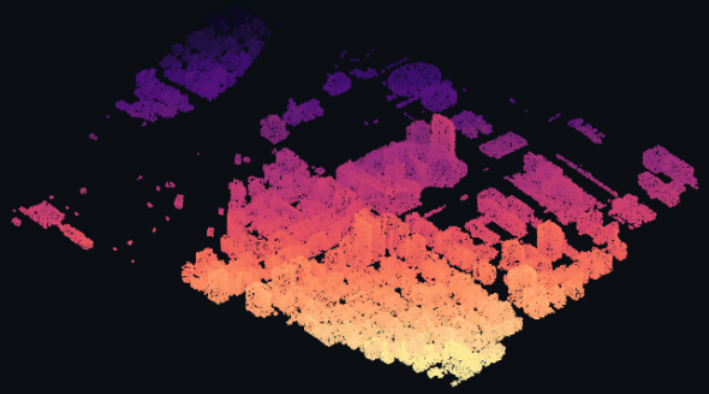
革新と方法

光の方向ごとに、シーンを 2 枚のデプスバッファへ書き込む

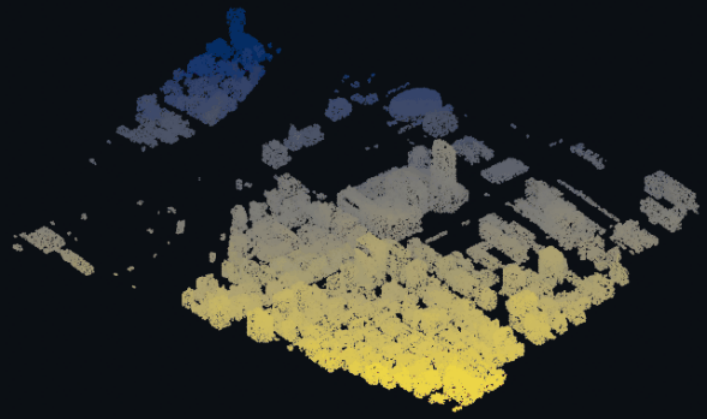
1 面あたりの問い合わせは、
遮蔽物が何棟あっても
1 回の参照。
その 1 パスを街区の
全面で使い回します。

背面バッファ（光に背を向けた面）
が街区の影を解き、前面バッファ（光を向いた面）
が重なり合う立体に埋もれた面を見つけます。
面の向きで分けるので、自己遮蔽が起きません。

背面バッファ（街区の影）

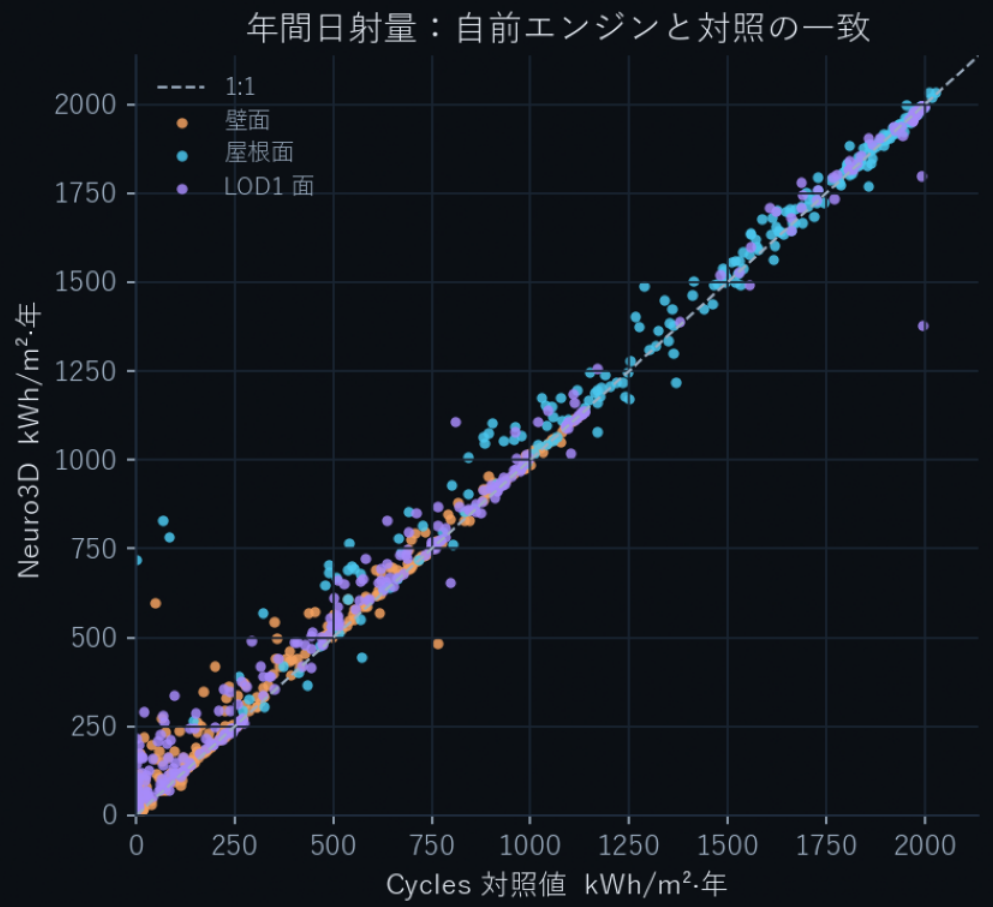


前面バッファ（埋もれた面）



科学的根拠：対照との一致

Blender Cycles（パストレーシング）を同一機・同一条件で



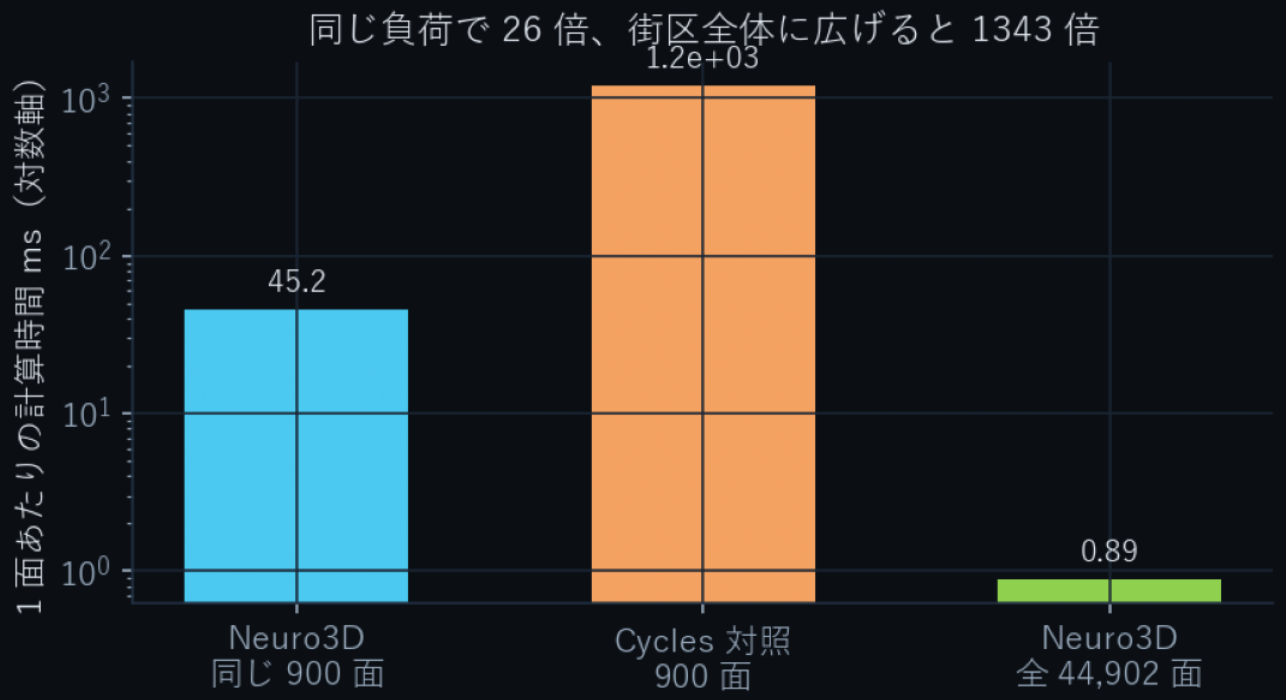
面の種類別（開けた 772 面）

屋根面	中央値 1.2 %	n=250
壁面	中央値 6.3 %	n=233
LOD1 面	中央値 4.0 %	n=289

R^2 0.985、相対誤差の中央値 2.6%。
対照の校正誤差は解析解に対して 2.2e-06
で、対照そのものが正しいことを先に確かめています。

結果：計算コスト

同じ機械、同じ方向、同じ面で測った比較



同じ 900 面・同じ 286 方向

26 倍

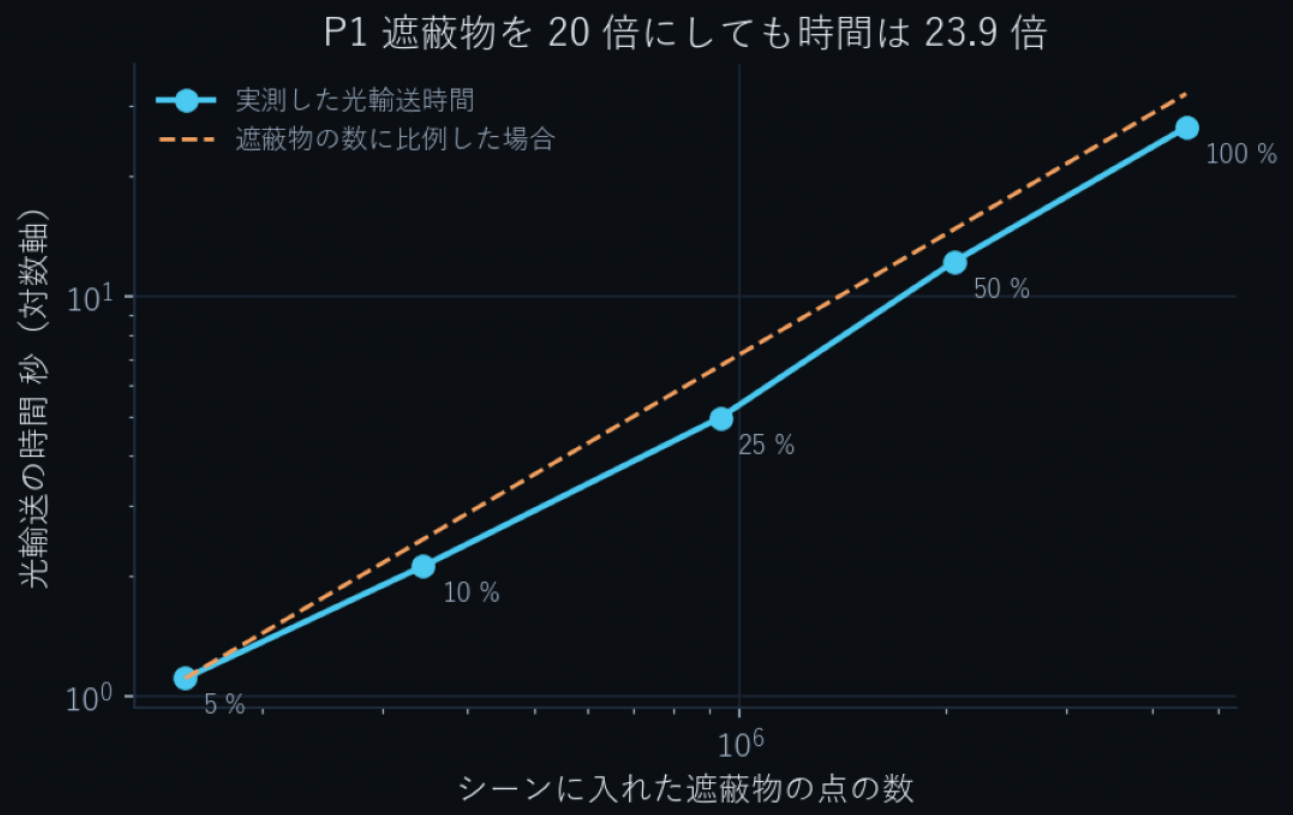
街区全体 44,902 面に広げると

1 面あたり 1343 倍

1 方向のコストが面の数にほぼ依存しないので、
街区を広げるほど差が開きます。前処理は計測に含めていません。
GPU では測っていないので主張しません。

新しい検証 ①　スケーリング

クエリ面を固定したまま、遮蔽物だけを 20 倍にする



遮蔽物の点　×29.0

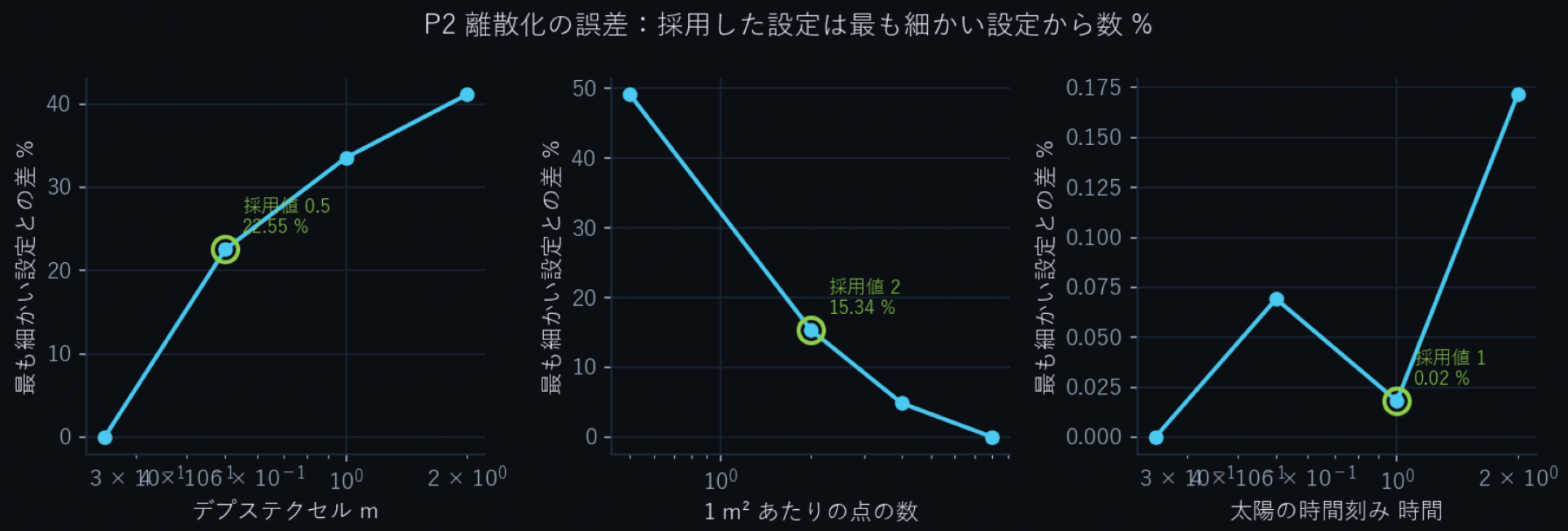
時間　×23.9

両対数の傾き 0.94

散布書き込みはシーンの点数にほぼ比例します。
遮蔽物の数に依存しないのは 1 面あたりの問い合わせのほうで、
その 1 パスが街区の全面に使い回されるのが速さの源です。
以前の版の書き方は、この実測に合わせて訂正しました。

新しい検証 ②　数値の収束

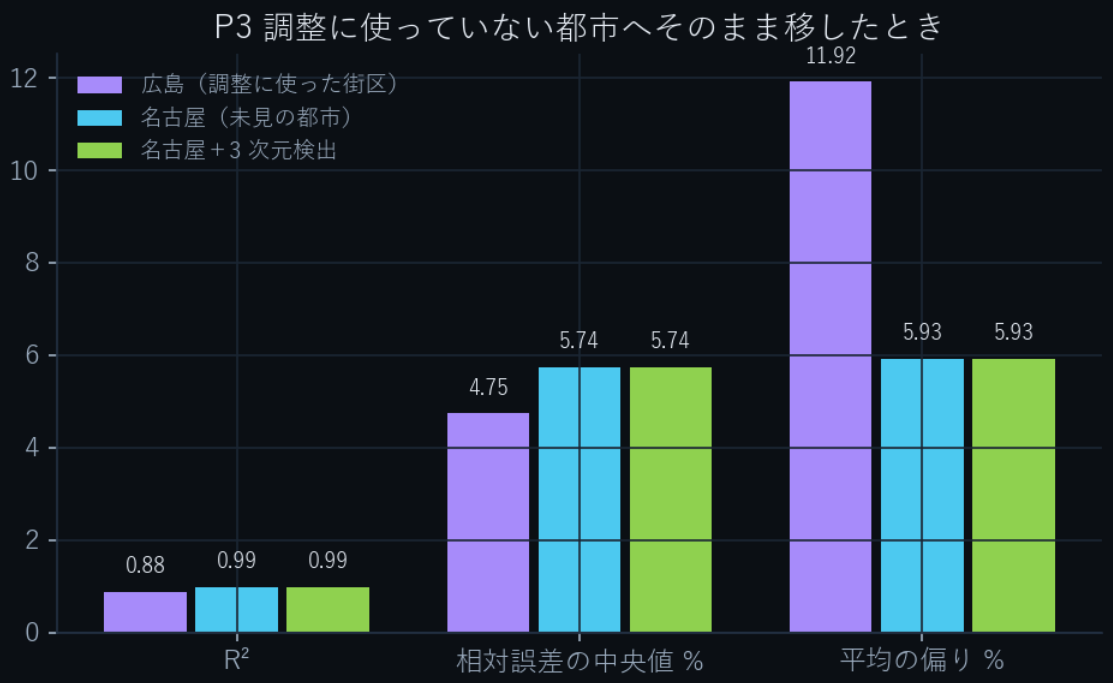
設定を変えたとき、答えがどれだけ動くか

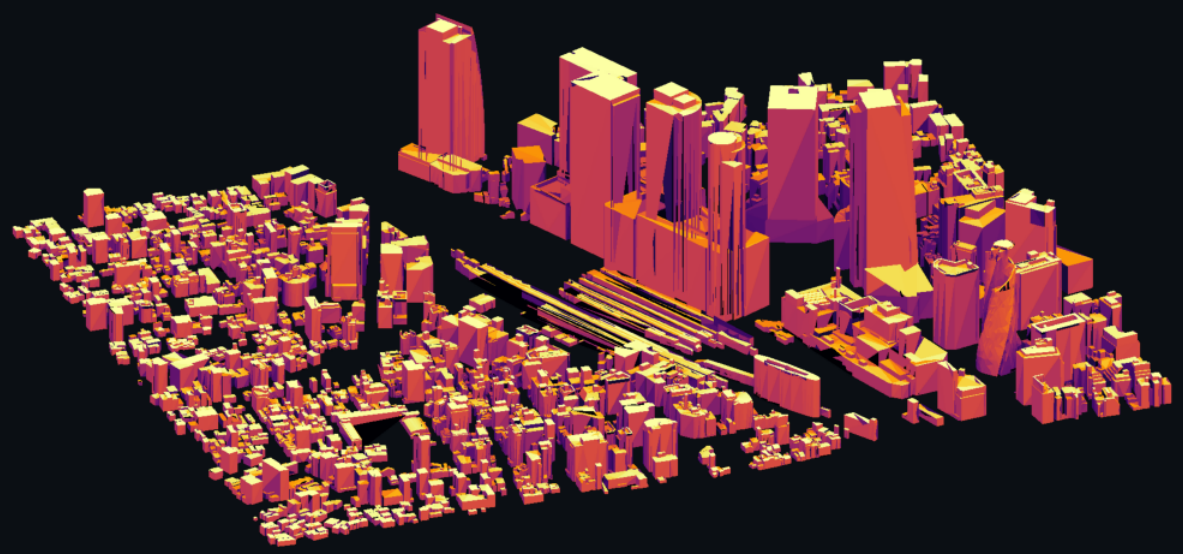


採用した設定（テクセル 0.5 m、1 m² あたり 2 点、1 時間刻み）は、それぞれの最も細かい設定に対して texel_m で 22.55 %、samples_per_m2 で 15.34 %、hour_step_h で 0.02 %。離散化による誤差は、対照との差よりずっと小さいことがわかります。

新しい検証 ③　別の都市へ

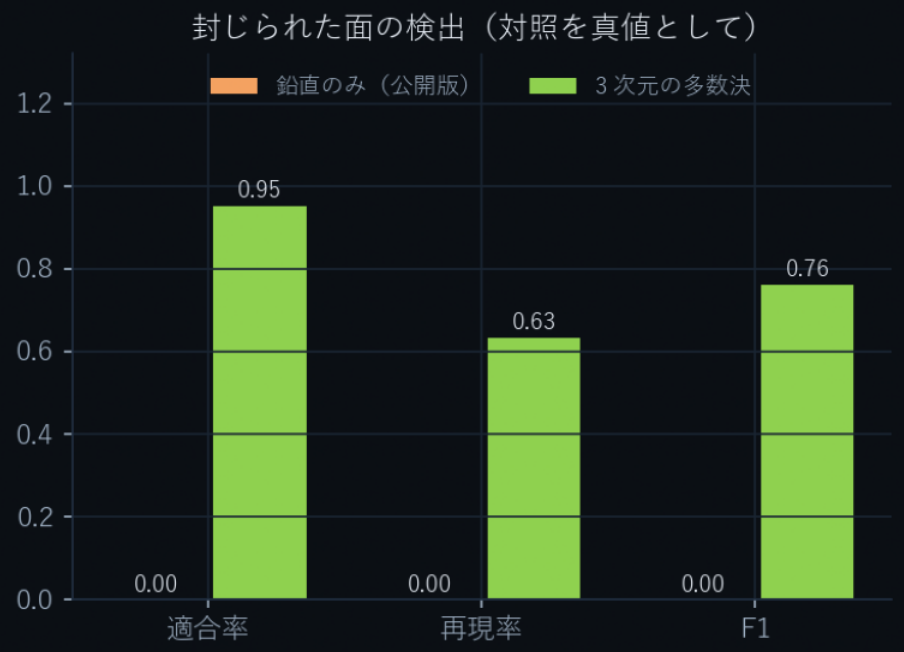
調整に一度も使っていない名古屋市の街区で、そのまま測り直す
名古屋市 3 次メッシュ 52366700 ／ 設定は一切変えず ／ R^2 0.986、相対誤差の中央値 5.7 %





新しい検証 ④　埋もれた面の検出

鉛直の判定を 3 次元の多数決に置き換える

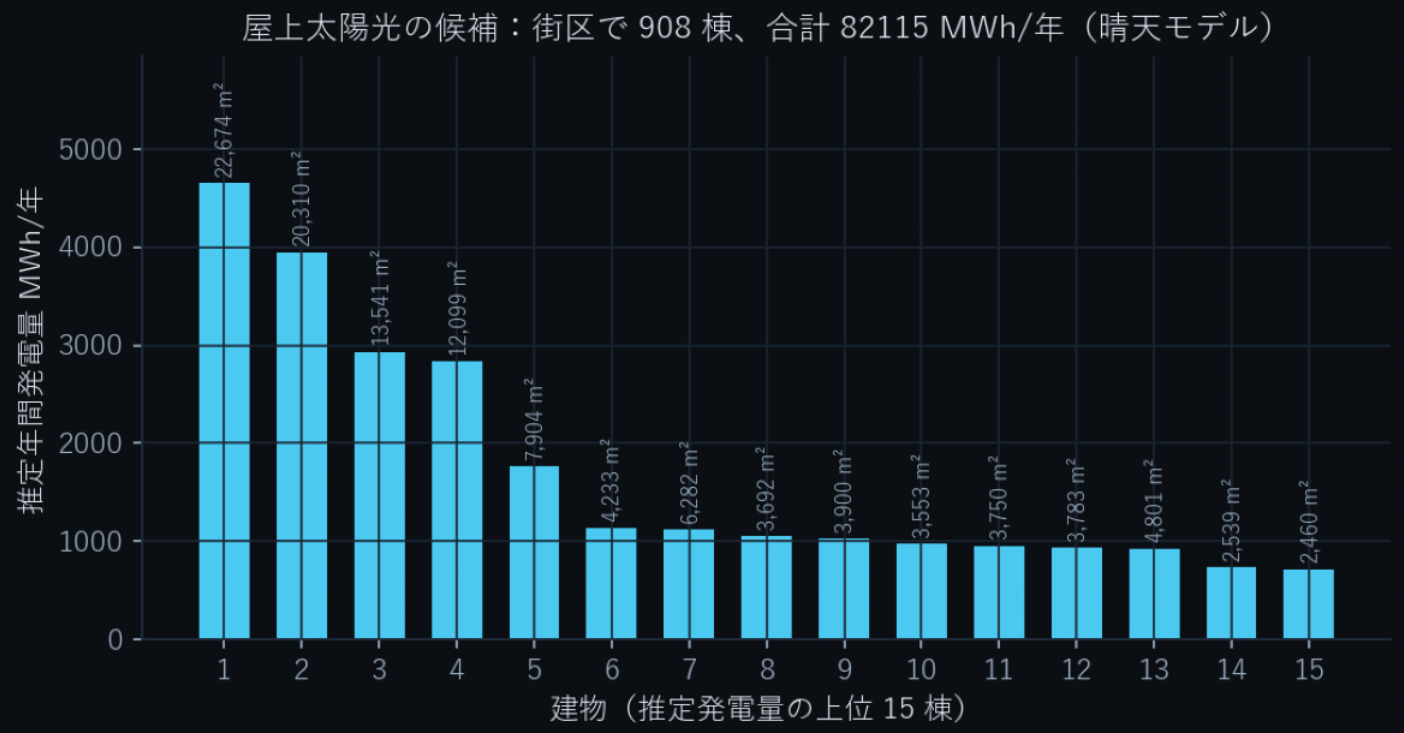




6 方向の座標軸と面の法線、計 7 本の光線のうち 5 本以上で内部と出たら「封じられた面」としました。基準は測る前に決めました。検出は 1924 面から 6008 面へ増え、再現率は 0.00 から 0.63 へ。この検出器で事前条件を測り直すと R^2 は 0.989 になります（登録時の判定はそのまま残しています）。

応用と影響

面ごとの数値を、自治体が次に見に行くべき屋根の一覧に変える



908

候補の棟数

82,115

推定 MWh/年

屋根面のうちパネルが載る傾きのものを棟ごとに集計し、
モジュール効率 20%・システム出力係数
0.80 で年間発電量を推定します。
数字は CSV で出力され、係数は差し替えられます。
晴天モデルによる推定で、現地調査でも実測でもありません。

解析解による単体試験

データなしで走り、CI で毎回実行されます

試験	解析値	実測	差
T1 遮蔽なしの地面の天空率	1.000	1.000	0.000
T2 高い壁の足元の天空率	0.500	0.519	+0.019
T3 箱の影／日なた／屋根（4 点）	0 / 1 / 1 / 1	一致	—
T4 正方形中庭の中心の天空率	0.554	0.588	+0.034

解析解のある 4 つの配置で、影そのものが正しいことを確かめています。

別の 4 件（selftest_sealed.py）では、封じられた面の検出を合成形状で検証：
屋根のない箱の中の箱を、鉛直判定は 0/12、3 次元の多数決は 12/12
で見つけます。T2 と T4 の差は原因を特定できておらず、「観察事項と限界」
に残しています。

再現できること

公開しているのは結果ではなく、結果に至る道です

- PLATEAU の ZIP 全体（1 GB 以上）は落とさず、HTTP レンジ要求で必要な 1 タイルだけを取得します。
- 対照の Blender は自動で取得し、同じ機械で走ります。
- 失敗した反復も含め、全履歴を experiments/LOG.md に残しています。
- 4 つの解析試験は毎コミット CI で実行されます。
- 元データは再配布せず、取得スクリプトのみを公開しています。

```
python code/run_pilot.py --tile ... --normals ... --outdir results
```

リポジトリ

ビューア

結論

街区をまるごと、速く、確かめられるかたちで

- PLATEAU の街区 44,902 面を 40 秒で解き、パストレーサと R^2 0.985 で一致します。
- 調整に使っていない別の都市でも、設定を変えずに同じ精度が出ます。
- 出力はそのまま屋上太陽光の候補一覧になり、ブラウザで誰でも触れます。
- コード、対照、失敗した反復まで公開し、1 つのコマンドで再現できます。
- 通らなかった条件と原因不明の差は、次ページにそのまま残しています。

ビューア

リポジトリ

観察事項と限界

通らなかった条件も、原因のわからない差も、そのまま載せます

- 事前登録した精度条件 H1 は、登録時の構成では全 900 面で通っていません（R^2 0.884、偏り +11.9 %、95 パーセンタイル 57265 %）。

  3 次元検出器では R^2 0.989・偏り +4.2 %・中央値 2.5 % まで改善しますが、それは契約後の測定です。

  95 パーセンタイルの条件は 2160 % で、どちらの構成でも満たしていません（対照がほぼ 0 の面で比率が発散）。
- 食い違う 128 面のうち 81 面は封じられた面と確認（適合率 0.95）。残る 47 面の原因は特定できていません。
- 1 方向のコストはシーンの点数にほぼ比例します（両対数の傾き 0.943）。依存しないのは 1 面あたりの問い合わせです。
- T2・T4 の解析解との差（+0.019、+0.034）は原因を特定できていません。
- 晴天モデルによる計算値で、実測気象データではありません。相互反射・植生・地形は含めていません。
- 「事前登録」は自己申告です。GPU では測っていないので GPU の速さは主張しません。
- 第三者による独立監査は未了です。

数値はすべて公開リポジトリの results/ と experiments/ から追えます。